ARC-AGI-2 LLM Serving Audit

README — ARC-AGI-2 LLM Serving Pre-GPU Audit

Report date: 2026-10-25
Language: বাংলা (flags, errors, versions, URLs in English)
Purpose: GPU quota খরচ করার **আগে** 4× L4 PCIe-only serving configuration-এর correctness এবং reliability audit।
Main decision: NO-GO for unvalidated speculative production; GO for bounded, gated preflight experiments.

> **URL FORMAT:** এই README-র সব evidence link নথির শেষের **FULL SOURCE URL REGISTER**-এ সম্পূর্ণ `https://...` আকারে আছে। Markdown label-only link ব্যবহৃত হয়নি; Claude/GPT যেন সরাসরি URL copy/search করতে পারে। PDF-তেও পূর্ণ URL অক্ষরে দেখা যাবে।

> **Scope & caution:** এটি পূর্ববর্তী ChatGPT serving-audit-এর reproducible handoff। 2026-10-25-এ প্রধান upstream issue (#44145, #58807, #36548, #53912, #36889, #38009, #39597, #62304, #58241) public issue pages দিয়ে পুনঃযাচাই করা হয়েছে। বাকি source registry-র সব URL একই সেশনে পুনরায় independently verified হয়নি। **Source exists** আর **fix is merged into pinned wheel** এক নয়। Production patch আনার আগে actual wheel commit/PR diff match করতে হবে।

Exact environment / comparison rules

- Kaggle, offline at runtime; **4× NVIDIA L4 24GB**, Ada **sm_89**, CUDA driver 13.0, PCIe PHB only, no NVLink; measured GPU P2P 5.5× slower than SHM; TP4 custom all-reduce disabled; NCCL falls back to another supported transport (exact SHM/P2P profiling still required).
- Python 3.12, glibc 2.35; target Qwen3.8-27B hybrid Gated DeltaNet + full attention + MTP; generated outputs 60k–90k, total context cap approximately 155k, 12–24 concurrent streams.
- Sampling: temperature 1.0, top_p 0.95, top_k 20. Streaming clients via SGLang `/generate` with `input_ids` or vLLM OpenAI interface.
- Weight candidates: RedHatAI INT4 W4A16/group128/compressed-tensors/Marlin/AWQ smooth with BF16 MTP; official-layout block-128 FP8 repack including MTP; z-lab Qwen3.8-27B-DFlash2 BF16.
- **Engine A:** SGLang 0.5.21; torch 2.13.0 cu130; flashinfer 0.6.18; sglang-kernel 0.4.7; transformers 5.12.1.
- **Engine B:** vLLM 0.32.0; torch 2.13.0 cu130; flashinfer 0.7.0.post1; transformers 5.19.0; compressed-tensors 0.17.0; xgrammar 0.2.8; triton 3.7.1.

Critical benchmark rule: INT4 vs FP8 বা BF16 target token IDs exact same হওয়া আবশ্যক নয়। Speculative correctness comparison করতে হবে **একই quantized checkpoint + একই kernel/backend/cache settings**-এর target-only বনাম speculative path দিয়ে। Temperature 0 greedy is the identity diagnostic; temperature 1.0/ top_p .95/ top_k 20-এ RNG consumption আলাদা হওয়ায় byte-identical completions বাধ্যতামূলক নয়। তখন distribution/task-level correctness পরীক্ষা প্রয়োজন।

Highest-priority silent-accuracy risks (FIRST)

1. **SGLang DFlash2 concurrency corruption:** apparent normal output wrong; upstream #36548: serial 0/100, concurrent 111/304 in one report; `fp32`, graph-disabled, width4 each still 37/304. **Not a proven L4 rate**. Source S04.

2. **SGLang DFlash thinking divergence:** upstream #38009 greedy first difference at token 30 with thinking enabled, no-thinking control matched; different RTX 3090 environment, not a pinned-L4 reproduction. Source S05.
3. **SGLang FlashInfer autotune restart nondeterminism:** #39597, `--disable-flashinfer-autotune` removes it in reported 8/8 restart test; your command already includes the guard, but this does **not** fix independent corruption. Source S06.
4. **SGLang speculative Mamba state boundary:** #43029 committed-length boundary fix post-v0.5.21; candidate for targeted backport. Source S03.
5. **vLLM hybrid MTP + prefix caching:** #53912 reports silent repeated punctuation, empty outputs with normal HTTP/finish reason. Turning cache off is safest first run. Source V01.
6. **vLLM GDN state commit:** #62304 demonstrates `ngram` speculative wrong greedy outputs, **not direct MTP evidence**, but validates the need for per-method recurrent-state checks. Source V04.
7. **Mamba/cache capacity silently caps running:** #36889 shows `max_running_requests` accepted while effective concurrency is smaller. Throughput can misleadingly look hardware-limited. Source S07.

How to detect silent failures

- 30–50 fixed prompts; test `temperature=0`, `top_p=1.0`, **identical** model, tokenizer, chat template, checkpoint, backend and seed.
- Compare **generated token IDs** for target-only vs speculative, not just rendered text; report `first_diff_index`, accepted/rejected draft counts and per-request metadata.
- Run serial, 4/8/12/16/20/24 concurrent, 3 clean restarts; both thinking on/off; cache cold/warm; chunked prefill and >64k contexts.
- Force speculative all-reject / partial-accept / full-accept paths; compare state commit/rollback and resumed output.
- Detect repeated punctuation/CJK, unexpected empty output, early `finish_reason`, malformed tools, duplicated/truncated streaming tokens, target-only benchmark deviations.

- **Fail closed:** any reproducible incorrect greedy result or unexplained cross-request contamination disqualifies an arm until fixed. A slight numerical logit margin difference is not proof of corruption by itself; investigate with a BF16/logits reference.

Configuration summary — all 15 required arms

P0 = potential silent-accuracy/state issue; P1 = crash/OOM/capacity blocker; P2 = slowdown/memory efficiency. Evidence IDs expand to full URLs below. Confidence distinguishes upstream evidence from your exact stack.

Config	Issue (crash / wrong output / slowdown / memory)	Evidence URL IDs	Affects pinned version / fix status	Severity	Fix / workaround	Confidence
S1 - SGLang target-only	Batch/backend numerical drift; long-context KV/ OOM; no speculation reference	S15, S16	0.5.21 exact Qwen3.8+L4 regression not established	P1; numerical can be P0 if task correctness changes	Keep `--linear-attn-backend triton`; same-checkpoint greedy serial/concurrent; 2048 vs 4096 prefill A/B	Medium risk; low exact-version evidence
S2 - INT4 EAGLE MTP steps3/ draft4	Duplicate BF16 MTP embed/head starves target KV pool; speculative committed-state boundary	S01, S02, S03	#44145 explicitly reproduced on SGLang v0.5.21; #43029 integrated after 0.5.21	P0 correctness / P1 memory	Backport reviewed #43029 and #45151 where applicable; test shared embedding/head and KV capacity; otherwise target-only	High upstream; medium checkpoint-specific
S3 - MTP steps2/ draft3; fraction .86; ReplaySSM fp16/ fp32	Mamba state mismatch, cache capacity cap, OOM at increased static fraction	S03, S07, S14	Committed-boundary fix after stock v0.5.21; ReplaySSM exact version regressions not established	P0/P1	One factor at a time; `--mamba-ssm-dtype float32` control; verify `--enable-linear-replayssm-spec` compatibility with chosen verify backend; keep .83 fallback	Medium; ReplaySSM-specific low
S4 - Competitor FP8/ INT4 EAGLE flag bundle	State pool starvation; backend/ precision changes; scoreboard claim not transferable	S13, S01, S07	Exact LB result unverified; 0.5.21 subset may load with different memory and numeric behavior	P0 if wrong state; P1 cap	A/B one at a time: FP8 target-only, FP8 MTP; `extra_buffer` vs `extra_buffer_lazy`; explicit Mamba cache floor; INT4 separately	Medium risk; low expected-score confidence
S5 - DFlash2 at 20/24 streams (mrr21/25)	Wrong normal-looking concurrent outputs, admission cap, graph-buffer size mismatch, OOM	S04, S07, S08, S09	Your local 12->13 tensor mismatch confirmed; public corruption on other GPU/build; backports did not solve admission in your test	P0/P1	Decode graph batch max >= mrr; inspect effective running, not configured; `--min-free-slots-delay 1` is not a correctness fix; fallback DFlash12/target-only	High local crash; medium corruption applicability
S6 - Best MTP at 20 streams	MTP KV starvation, state commit mismatch, graph capture / state cap	S01, S02, S03, S17	Same affected MTP subsystem; 20-way L4 result not found	P0/P1	Only after S2/S3 correctness and capacity gates; ramp 8->12->16->20; graph-disabled diagnostic	Medium

Config	Issue (crash / wrong output / slowdown / memory)	Evidence URL IDs	Affects pinned version / fix status	Severity	Fix / workaround	Confidence
S7 - DFlash width4 vs width8	8-position-trained drafter runtime block-size override; accept length may collapse; width reduction does not fully fix correctness	S11, S04, S10, M01, M02	Checkpoint config block_size=8; runtime width override is testable; S04 report width4 still wrong	P0 for corruption; P2 speed	Compare width 4,6,8 with same checkpoint; track warnings, acceptance, greedy divergence and real tokens/sec	High checkpoint detail; medium exact L4 risk
S8 - GDN backend flashinfer or cutedsl on sm_89	GDN backend hardware and phase support mismatch; likely unsupported path/fallback	S22, S14	Current support guidance differs from full-attention backend; exact v0.5.21 fallback needs startup check	P1	Reference `--linear-attn-backend triton`; separately verify prefill/decode/ verify backend help flags before changing	Medium-high docs; low exact fallback
S9 - Two independent TP2 DFlash6/ MTP, 8 each	Per-replica weights, KV state and CUDA graph overhead; NCCL/ports; unknown aggregate throughput	S12, S18, N01, N02	TP4 PCIe custom all-reduce disabled in your logs; TP2 memory fit not measured	P1 memory; P2 performance	Separate CUDA_VISIBLE_DEVICES, --port and --nccl-port; check GPU pairing and A/B memory; 8+8 bounded probe	High operational; low speed gain
V1 - vLLM FP8 MTP3, 16 seq, fp8 KV, prefix cache	GDN/MTP with prefix cache silently returns punctuation/ empty content; FP8 KV is not proven fix	V01, V12, V06	#53912 reproduced 0.28; status of general fix in 0.32 not proven	P0	`--no-enable-prefix-caching` on first MTP run; cache hit/miss regression; `--kv-cache-dtype auto` diagnostic	High upstream; medium 0.32 exact
V2 - vLLM MTP spec2/3/4	Acceptance/state commit divergence, EOS/repetition; MTP-width load/ graph instability	V01, V04, V10	#62304 affects ngram not MTP; do not claim it directly proves MTP error	P0 if corruption; P2 speed	`--speculative-config {"method":"mtp","num_speculative_tokens":2}` then 3 then 4; cache disabled; matched target-only oracle	Medium concern; specific MTP low
V3 - vLLM INT4 W4A16 compressed-tensors + BF16 MTP	Weight loader expects packed MTP weights but separate BF16 MTP shard supplies plain .weight; boot crash	V02, V03	vLLM v0.30 demonstrated; PR pending in source seen; RedHatAI exact checkpoint not reproduced	P1 BLOCKER	Inspect safetensors key names and quantization ignore/ exclusion; reviewed loader patch or disable MTP on INT4	High family; medium checkpoint
V4 - max_num_seqs16/20/24	Concurrency OOM, latency regression; GDN/cache state errors can scale with load	V01, V09	No measured safe threshold for 4xL4 + 155k	P1; P0 with prefix cache corruption	16 first, then 20 and 24; first with caching disabled; `--enforce-eager` only graph diagnostic	Medium
V5 - vLLM DP2 TP2	Replica VRAM/RAM, GPU mapping, DP routing/port conflicts; apparent total capacity overestimated	V13, V14, N01	MoE EP bug #50681 is NOT direct evidence for dense Qwen3.8 DP2	P1 memory; P2 throughput	`--data-parallel-size 2 --tensor-parallel-size 2 --max-num-seqs 8` initial; verify rank routing and aggregate tok/s; compare independent TP2 servers	High semantics; low exact failure
V6 - SM89 FP8 block128 Triton/ default/ tuned/ Marlin	CUTLASS blockwise-FP8 path excludes SM89; fallback may pick Marlin; tuning could be faster for certain GEMM sizes	V05, V15, V16	SM89 confirmed upstream; which exact kernel chosen in 0.32.0 must be read from logs	P2 perf; P0 only if measured numeric corruption	Compare actual selected default against optional `VLLM_TEST_FORCE_FP8_MARLIN=1` after env support check and tuned Triton; benchmark same logits	High hardware applicability

Arm-by-arm exact evidence URLs

এখানে hidden clickable labels নেই; প্রতিটি source-ই raw, full URL। Sources প্রযোজ্য বলেই আপনার exact pinned wheel-এ failure confirmed নয়।

S1 - SGLang target-only

- **Issue:** Batch/backend numerical drift; long-context KV/OOM; no speculation reference
- **Pinned applicability:** 0.5.21 exact Qwen3.8+L4 regression not established
- **Severity:** P1; numerical can be P0 if task correctness changes
- **Exact workaround / validation:** Keep `--linear-attn-backend triton`; same-checkpoint greedy serial/concurrent; 2048 vs 4096 prefill A/B
- **Confidence:** Medium risk; low exact-version evidence
- **Evidence (raw URLs):**
  - S15: https://github.com/sgl-project/sglang/issues/43956
  - S16: https://github.com/sgl-project/sglang/issues/22087

S2 - INT4 EAGLE MTP steps3/draft4

- **Issue:** Duplicate BF16 MTP embed/head starves target KV pool; speculative committed-state boundary
- **Pinned applicability:** #44145 explicitly reproduced on SGLang v0.5.21; #43029 integrated after 0.5.21
- **Severity:** P0 correctness / P1 memory
- **Exact workaround / validation:** Backport reviewed #43029 and #45151 where applicable; test shared embedding/head and KV capacity; otherwise target-only
- **Confidence:** High upstream; medium checkpoint-specific
- **Evidence (raw URLs):**
  - S01: https://github.com/sgl-project/sglang/issues/44145
  - S02: https://github.com/sgl-project/sglang/pull/45151
  - S03: https://github.com/sgl-project/sglang/pull/43029

S3 - MTP steps2/draft3; fraction .86; ReplaySSM fp16/fp32

- **Issue:** Mamba state mismatch, cache capacity cap, OOM at increased static fraction
- **Pinned applicability:** Committed-boundary fix after stock v0.5.21; ReplaySSM exact version regressions not established
- **Severity:** P0/P1
- **Exact workaround / validation:** One factor at a time; `--mamba-ssm-dtype float32` control; verify `--enable-linear-replayssm-spec` compatibility with chosen verify backend; keep .83 fallback
- **Confidence:** Medium; ReplaySSM-specific low
- **Evidence (raw URLs):**
  - S03: https://github.com/sgl-project/sglang/pull/43029
  - S07: https://github.com/sgl-project/sglang/issues/36889
  - S14: https://github.com/sgl-project/sglang/pull/34592

S4 - Competitor FP8/INT4 EAGLE flag bundle

- **Issue:** State pool starvation; backend/precision changes; scoreboard claim not transferable
- **Pinned applicability:** Exact LB result unverified; 0.5.21 subset may load with different memory and numeric behavior
- **Severity:** P0 if wrong state; P1 cap
- **Exact workaround / validation:** A/B one at a time: FP8 target-only, FP8 MTP; `extra_buffer` vs `extra_buffer_lazy`; explicit Mamba cache floor; INT4 separately
- **Confidence:** Medium risk; low expected-score confidence
- **Evidence (raw URLs):**
  - S13: https://github.com/sgl-project/sglang/issues/41938
  - S01: https://github.com/sgl-project/sglang/issues/44145
  - S07: https://github.com/sgl-project/sglang/issues/36889

S5 - DFlash2 at 20/24 streams (mrr21/25)

- **Issue:** Wrong normal-looking concurrent outputs, admission cap, graph-buffer size mismatch, OOM

- **Pinned applicability:** Your local 12->13 tensor mismatch confirmed; public corruption on other GPU/build; backports did not solve admission in your test
- **Severity:** P0/P1
- **Exact workaround / validation:** Decode graph batch max >= mrr; inspect effective running, not configured; `--min-free-slots-delay 1` is not a correctness fix; fallback DFlash12/target-only
- **Confidence:** High local crash; medium corruption applicability
- **Evidence (raw URLs):**
  - S04: https://github.com/sgl-project/sglang/issues/36548
  - S07: https://github.com/sgl-project/sglang/issues/36889
  - S08: https://github.com/sgl-project/sglang/pull/41872
  - S09: https://github.com/sgl-project/sglang/pull/44401

S6 - Best MTP at 20 streams

- **Issue:** MTP KV starvation, state commit mismatch, graph capture / state cap
- **Pinned applicability:** Same affected MTP subsystem; 20-way L4 result not found
- **Severity:** P0/P1
- **Exact workaround / validation:** Only after S2/S3 correctness and capacity gates; ramp 8->12->16->20; graph-disabled diagnostic
- **Confidence:** Medium
- **Evidence (raw URLs):**
  - S01: https://github.com/sgl-project/sglang/issues/44145
  - S02: https://github.com/sgl-project/sglang/pull/45151
  - S03: https://github.com/sgl-project/sglang/pull/43029
  - S17: https://github.com/sgl-project/sglang/issues/37052

S7 - DFlash width4 vs width8

- **Issue:** 8-position-trained drafter runtime block-size override; accept length may collapse; width reduction does not fully fix correctness
- **Pinned applicability:** Checkpoint config block_size=8; runtime width override is testable; S04 report width4 still wrong

- **Severity:** P0 for corruption; P2 speed
- **Exact workaround / validation:** Compare width 4,6,8 with same checkpoint; track warnings, acceptance, greedy divergence and real tokens/sec
- **Confidence:** High checkpoint detail; medium exact L4 risk
- **Evidence (raw URLs):**
  - S11: https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/speculative/dflash_worker_v2.py
  - S04: https://github.com/sgl-project/sglang/issues/36548
  - S10: https://github.com/sgl-project/sglang/issues/36127
  - M01: https://huggingface.co/incoai/Qwen3.8-27B-DFlash2
  - M02: https://huggingface.co/z-lab/Qwen3.8-27B-DFlash2

S8 - GDN backend flashinfer or cutedsl on sm_89

- **Issue:** GDN backend hardware and phase support mismatch; likely unsupported path/fallback
- **Pinned applicability:** Current support guidance differs from full-attention backend; exact v0.5.21 fallback needs startup check
- **Severity:** P1
- **Exact workaround / validation:** Reference `--linear-attn-backend triton`; separately verify prefill/decode/verify backend help flags before changing
- **Confidence:** Medium-high docs; low exact fallback
- **Evidence (raw URLs):**
  - S22: https://github.com/sgl-project/sglang/blob/main/docs/docs/advanced_features/attention_backend.mdx
  - S14: https://github.com/sgl-project/sglang/pull/34592

S9 - Two independent TP2 DFlash6/MTP, 8 each

- **Issue:** Per-replica weights, KV state and CUDA graph overhead; NCCL/ports; unknown aggregate throughput
- **Pinned applicability:** TP4 PCIe custom all-reduce disabled in your logs; TP2 memory fit not measured
- **Severity:** P1 memory; P2 performance

- **Exact workaround / validation:** Separate CUDA_VISIBLE_DEVICES, --port and --nccl-port; check GPU pairing and A/B memory; 8+8 bounded probe
- **Confidence:** High operational; low speed gain
- **Evidence (raw URLs):**
  - S12: https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/distributed/device_communicators/custom_all_reduce_utils.py
  - S18: https://github.com/sgl-project/sglang/issues/37215
  - N01: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html
  - N02: https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md

V1 - vLLM FP8 MTP3, 16 seq, fp8 KV, prefix cache

- **Issue:** GDN/MTP with prefix cache silently returns punctuation/empty content; FP8 KV is not proven fix
- **Pinned applicability:** #53912 reproduced 0.28; status of general fix in 0.32 not proven
- **Severity:** P0
- **Exact workaround / validation:** `--no-enable-prefix-caching` on first MTP run; cache hit/miss regression; `--kv-cache-dtype auto` diagnostic
- **Confidence:** High upstream; medium 0.32 exact
- **Evidence (raw URLs):**
  - V01: https://github.com/vllm-project/vllm/issues/53912
  - V12: https://github.com/vllm-project/vllm/pull/48375
  - V06: https://github.com/vllm-project/vllm/issues/61927

V2 - vLLM MTP spec2/3/4

- **Issue:** Acceptance/state commit divergence, EOS/repetition; MTP-width load/graph instability
- **Pinned applicability:** #62304 affects ngram not MTP; do not claim it directly proves MTP error
- **Severity:** P0 if corruption; P2 speed

- **Exact workaround / validation:** `--speculative-config {"method":"mtp","num_speculative_tokens":2}` then 3 then 4; cache disabled; matched target-only oracle
- **Confidence:** Medium concern; specific MTP low
- **Evidence (raw URLs):**
  - V01: https://github.com/vllm-project/vllm/issues/53912
  - V04: https://github.com/vllm-project/vllm/issues/62304
  - V10: https://github.com/vllm-project/vllm/issues/61738

V3 - vLLM INT4 W4A16 compressed-tensors + BF16 MTP

- **Issue:** Weight loader expects packed MTP weights but separate BF16 MTP shard supplies plain .weight; boot crash
- **Pinned applicability:** vLLM v0.30 demonstrated; PR pending in source seen; RedHatAI exact checkpoint not reproduced
- **Severity:** P1 BLOCKER
- **Exact workaround / validation:** Inspect safetensors key names and quantization ignore/exclusion; reviewed loader patch or disable MTP on INT4
- **Confidence:** High family; medium checkpoint
- **Evidence (raw URLs):**
  - V02: https://github.com/vllm-project/vllm/issues/58807
  - V03: https://github.com/vllm-project/vllm/pull/58813

V4 - max_num_seqs16/20/24

- **Issue:** Concurrency OOM, latency regression; GDN/cache state errors can scale with load
- **Pinned applicability:** No measured safe threshold for 4xL4 + 155k
- **Severity:** P1; P0 with prefix cache corruption
- **Exact workaround / validation:** 16 first, then 20 and 24; first with caching disabled; `--enforce-eager` only graph diagnostic
- **Confidence:** Medium
- **Evidence (raw URLs):**
  - V01: https://github.com/vllm-project/vllm/issues/53912
  - V09: https://github.com/vllm-project/vllm/issues/55775

V5 - vLLM DP2 TP2

- **Issue:** Replica VRAM/RAM, GPU mapping, DP routing/port conflicts; apparent total capacity overestimated
- **Pinned applicability:** MoE EP bug #50681 is NOT direct evidence for dense Qwen3.8 DP2
- **Severity:** P1 memory; P2 throughput
- **Exact workaround / validation:** `--data-parallel-size 2 --tensor-parallel-size 2 --max-num-seqs 8` initial; verify rank routing and aggregate tok/s; compare independent TP2 servers
- **Confidence:** High semantics; low exact failure
- **Evidence (raw URLs):**
  - V13: https://docs.vllm.ai/en/latest/serving/data_parallel_deployment/
  - V14: https://github.com/vllm-project/vllm/issues/50681
  - N01: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html

V6 - SM89 FP8 block128 Triton/default/tuned/Marlin

- **Issue:** CUTLASS blockwise-FP8 path excludes SM89; fallback may pick Marlin; tuning could be faster for certain GEMM sizes
- **Pinned applicability:** SM89 confirmed upstream; which exact kernel chosen in 0.32.0 must be read from logs
- **Severity:** P2 perf; P0 only if measured numeric corruption
- **Exact workaround / validation:** Compare actual selected default against optional `VLLM_TEST_FORCE_FP8_MARLIN=1` after env support check and tuned Triton; benchmark same logits
- **Confidence:** High hardware applicability
- **Evidence (raw URLs):**
  - V05: https://github.com/vllm-project/vllm/issues/58241
  - V15: https://github.com/vllm-project/vllm/pull/56085
  - V16: https://github.com/vllm-project/vllm/issues/56493

Version and model architecture caveats

- #44145 explicitly uses SGLang v0.5.21 and untied BF16 MTP embeddings, but the original reported memory loss (KV 225,211 to 15,363 tokens) is a particular setup, **not the predicted TP4 L4 number**. Review #45151 patch and test when duplicate head buffers are released relative to `init_target_memory_pool()`.
- #58807 is vLLM v0.30.0 and a W4A16 checkpoint with dense BF16 MTP; the exact checkpoint differs from RedHatAI's. Static safetensors/config audit first; do not claim the same boot crash without reproducing.
- #62304 applies specifically to ngram speculation on hybrid GDN; it is **not** proof that every MTP configuration is broken.
- #53912 saw FP8 KV reduce cache hits and corruption cases in one experiment. That is **not** equivalent to fixing corrupted prefix state; explicit `--no-enable-prefix-caching` is the safer reference.
- FP8, INT4, MTP and speculative verification need to be benchmarked for **accuracy AND throughput**. Comparing sampled text across different quantizations or kernel paths cannot isolate speculation.
- Every long-context limit concerns **prompt + output total tokens**. For example, context 155,648 does not allow 155,648 input plus 90,000 generated output simultaneously.

Hardware-specific risks: Ada sm_89 / L4 / PCIe

1. **Block FP8 GEMM kernel selection:** vLLM #58241 reports `cutlass_scaled_mm_supports_block_fp8` excludes SM89, potentially falling back to Marlin. Do not assume default Triton or best latency. Compare actual dispatch + matrix sizes. URLs: https://github.com/vllm-project/vllm/issues/58241 and https://github.com/vllm-project/vllm/pull/56085
2. **GDN backend limitations:** `--attention-backend flashinfer` for ordinary attention does not automatically make **linear GDN backend** supported on SM89. Keep `--linear-attn-backend triton` reference. Support details must be checked against installed v0.5.21 help/source. URL: https://github.com/sgl-project/sglang/blob/main/docs/docs/advanced_features/attention_backend.mdx

3. **PCIe collectives:** 4-way custom all-reduce disabled by your historical log. NCCL may still run via SHM/transport fallback. No validated 40–50ms comm cost without trace. Test small collectives 8KB–1MB, not only 64MB. URLs: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html and https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md
4. **FlashInfer JIT + offline notebooks:** wheel import alone does not prove all sm89 kernels/JIT artifacts are present. Cold-start smoke across prefill, decode, verify, CUDA graph before sealing offline Kaggle dataset. Relevant: https://github.com/vllm-project/vllm/issues/60262
5. **TP2 pairing:** 0/1 + 2/3 are proposed only. Choose pairing from `nvidia-smi topo -m` and `nvidia-smi topo -p2p p`. Separate process/CUDA visibility per replica.

Multi-server operational skeleton (independent SGLang TP2)

```
# Separate processes; add only your previously validated model/spec/cache flags.
CUDA_VISIBLE_DEVICES=0,1 NCCL_DEBUG=INFO \
python -m sglang.launch_server --model-path "$MODEL" \
  --tp-size 2 --host 127.0.0.1 --port 30000 --nccl-port 30001 \
  --max-running-requests 8 --mem-fraction-static 0.80

CUDA_VISIBLE_DEVICES=2,3 NCCL_DEBUG=INFO \
python -m sglang.launch_server --model-path "$MODEL" \
  --tp-size 2 --host 127.0.0.1 --port 31000 --nccl-port 31001 \
  --max-running-requests 8 --mem-fraction-static 0.80
```

Note: `.80` is a conservative experimental starting value, **not a verified winner**. Verify ports, NCCL process groups, GPU mapping, per-replica KV and Mamba capacity, memory after CUDA-graph capture, crash isolation and aggregate rather than single-replica throughput. Host-reserved/auto ports may vary by SGLang wheel; check help/output.

Recommended pre-GPU gates

Gate A - CPU/package/checkpoint static inspection

```
python - <<'PY'
import importlib.metadata as m
for name in ("sglang", "vllm", "torch", "transformers",
             "flashinfer-python", "sgl-kernel", "compressed-tensors",
             "triton", "xgrammar"):
    try: print(name, m.version(name))
    except m.PackageNotFoundError: print(name, "NOT INSTALLED")
import importlib.metadata as m
for name in ("sglang", "vllm", "torch", "transformers",
             "flashinfer-python", "sgl-kernel", "compressed-tensors",
             "triton", "xgrammar"):
    try: print(name, m.version(name))
    except m.PackageNotFoundError: print(name, "NOT INSTALLED")
PY

python -m sglang.launch_server --help > sglang_flags.txt
vllm serve --help > vllm_flags.txt
```

- Preserve wheel hashes/versions and model revision SHA256; check model `config.json`, `quantization_config`, safetensors index and dedicated BF16 MTP shard. Verify `mtp.fc.weight` vs `weight_packed` expectations before using vLLM INT4+MTP.
- Check #43029 and #45151 **actual applied commit** in your SGLang source: version string alone cannot tell you which local backports are present.
- Check vLLM exact MTP loader and PR #58813 before costly weight loading. Confirm `--no-enable-prefix-caching` spelling from vLLM 0.32 `--help`.
- Verify `VLLM_TEST_FORCE_FP8_MARLIN=1` exists and is consumed by your exact vLLM source before using it as A/B control.

Gate B - Safety acceptance criteria

Test	Required pass
Same-checkpoint temp=0 target vs spec	Identical token IDs; any first difference debugged with logits/state trace
Server restart x3	Greedy stable when stack identical
Serial and concurrent	No cross-request output/state contamination
Spec partial-rejection	Correct accepted-token committed state and next-token continuation
Prefix cache hot/cold	No malformed/repeated/empty output or unexpected token shift
Context 4k / 64k / 96k / 128k / near 155k	No OOM, runtime/index error, corruption, premature context failure
Streaming vs assembled	No token omission, duplication, reordering
Effective concurrency	Actual running reaches requested count or fails loudly with capacity limit

Test	Required pass
FP8/INT4 numeric A/B	Non-finite logits absent; task-score regression measured; not falsely requiring bitwise BF16 match

Gate C - Mamba admission fail-closed check

```
def check_capacity(configured_mrr: int, slots_per_request: int,
                   allocated_mamba_slots: int, effective_mrr: int) -> None:
    required = configured_mrr * slots_per_request
    if allocated_mamba_slots < required:
        raise RuntimeError(
            f"FAIL_MAMBA_CAPACITY: need={required}, allocated={allocated_mamba_slots}")
    if effective_mrr < configured_mrr:
        raise RuntimeError(
            f"FAIL_ADMISSION_CAP: configured={configured_mrr}, effective={effective_mrr}")
```

For DFlash **only if** same allocation rule as upstream #36889 holds: slots/request = `1 + speculative_num_draft_tokens`. Draft=6 -> 7 slots/request; `mrr=17` ->119, `mrr=21` ->147, `mrr=25` ->175. **These are conditional arithmetic floors, not verified measured cache requirements for Qwen3.8.** Use actual scheduler log/allocated Mamba pool and do not assume `4*mrr` is always sufficient.

Gate D - Kernel/communication and memory benchmark

- Log `nvidia-smi topo -m`, `nvidia-smi topo -p2p p`, `NCCL_DEBUG=INFO`, and NCCL transport selection.
- Run nccl-tests all-reduce 8KB–1MB (when binary already cached) for TP2 and TP4, and profile actual decode GPU critical-path. Do **not** sum overlapping four-GPU kernel times as wall time.
- Measure `engine_startup_sec`, `total_wall_sec`, `output_tokens`, true end-to-end tok/s, accepted tokens per verification, graph capture time, KV/Mamba slots, CUDA memory peak, first corrupted token, parser/tool-call integrity and failures.
- Repeated cold starts may cost substantial quota. Do smaller bounded probes before full ARC tasks.

High-value alternative arms not in original list

1. **DFlash6 and explicit actual Mamba admission profiling**: highest yield, because under-admission can imitate GEMM/NCCL bottleneck. Source S07.

2. **Target-only smaller batches, with exact workload**: compare 12/16/20/24 no-spec; a drafter does not automatically improve aggregate tokens/sec. Source N03.
3. **SGLang `extra_buffer_lazy` vs `extra_buffer` vs no radix cache** with equal prefixes/churn; source S13.
4. **2048 vs 4096 `--chunked-prefill-size`** across identical workloads; no guaranteed winner. Source S16.
5. **vLLM SM89 actual default dispatch vs tuned Triton block-FP8 vs Marlin** if profiler shows GEMM-bound. Source V05.
6. **SGLang 0.5.22 correctness branch** after controlled #43029 backport; **do not** assume 0.5.22 automatically closes all MTP/DFlash issues. Source S03.
7. **2×TP2** A/B with 8+8 requests; likely useful because TP4 all-reduce disabled, **not yet proven** faster for your exact stack. Source N01.

No public independent benchmark was confirmed for the **exact tuple** Qwen3.8-27B + 4× L4 PCIe-only + 155k context + 12–24 streams. Do not claim 2.4x, 550–650 tok/s, or LB 65.97 from other machine setups.

Run order and final GO/NO-GO

1. **CPU static**: wheel manifest, checkpoints, MTP BF16 exclusions, local patch presence; zero GPU cost.
2. **SGLang INT4 target-only**: GO to bounded correctness baseline (not immediate full competition).
3. **SGLang DFlash6 at 12 streams**: CONDITIONAL GO only after serial/concurrent greedy and true admission checks. `--disable-flashinfer-autotune` retain.
4. **SGLang MTP2/draft3 and MTP3/draft4**: NO-GO until #43029 state boundary and #44145/#45151 memory profiling assessed.
5. **SGLang DFlash at 20/24**: NO-GO until tensor graph-buffer + admission + corruption probes all pass.
6. **SGLang GDN FlashInfer / verify-only CuTe DSL on L4**: NO-GO as blind replacement for Triton.
7. **2×TP2**: GO to bounded capacity/profiling, never assume it is leaderboard-ready from startup alone.

8. **vLLM FP8 + MTP + prefix caching**: NO-GO for first reference; disable caching first.
9. **vLLM FP8 + MTP without prefix caching**: CONDITIONAL GO to bounded reference experiment.
10. **vLLM INT4 compressed-tensors + BF16 MTP**: NO-GO until loader shard layout compatibility checked/fixed.
11. **SM89 FP8 Marlin/Triton dispatch**: GO to kernel microbench plus task-level numerical/accuracy comparison.

FINAL: GPU quota ব্যয় করার আগে সর্বোচ্চ priority তিনটি: **(1) #43029 + #45151 backport status and pool sizing, (2) DFlash cross-request/think-state correctness, (3) Mamba effective admission.** Faster but wrong output = **NO-GO**.

FULL SOURCE URL REGISTER (explicit, copyable URLs)

Below: every evidence URL is printed in full, with no hidden linked label. Some entries are upstream documents/adjacent issues whose **exact pinned applicability is not yet verified**. Do not interpret inclusion as a confirmed bug or merged fix.

- **S01:** https://github.com/sgl-project/sglang/issues/44145
- **S02:** https://github.com/sgl-project/sglang/pull/45151
- **S03:** https://github.com/sgl-project/sglang/pull/43029
- **S04:** https://github.com/sgl-project/sglang/issues/36548
- **S05:** https://github.com/sgl-project/sglang/issues/38009
- **S06:** https://github.com/sgl-project/sglang/issues/39597
- **S07:** https://github.com/sgl-project/sglang/issues/36889
- **S08:** https://github.com/sgl-project/sglang/pull/41872
- **S09:** https://github.com/sgl-project/sglang/pull/44401
- **S10:** https://github.com/sgl-project/sglang/issues/36127
- **S11:** https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/speculative/dflash_worker_v2.py
- **S12:** https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/distributed/device_communicators/custom_all_reduce_utils.py
- **S13:** https://github.com/sgl-project/sglang/issues/41938

- **S14:** https://github.com/sgl-project/sglang/pull/34592
- **S15:** https://github.com/sgl-project/sglang/issues/43956
- **S16:** https://github.com/sgl-project/sglang/issues/22087
- **S17:** https://github.com/sgl-project/sglang/issues/37052
- **S18:** https://github.com/sgl-project/sglang/issues/37215
- **S19:** https://github.com/sgl-project/sglang/issues/35437
- **S20:** https://github.com/sgl-project/sglang/issues/26330
- **S21:** https://github.com/sgl-project/sglang/issues/25536
- **S22:** https://github.com/sgl-project/sglang/blob/main/docs/docs/advanced_features/attention_backend.mdx
- **V01:** https://github.com/vllm-project/vllm/issues/53912
- **V02:** https://github.com/vllm-project/vllm/issues/58807
- **V03:** https://github.com/vllm-project/vllm/pull/58813
- **V04:** https://github.com/vllm-project/vllm/issues/62304
- **V05:** https://github.com/vllm-project/vllm/issues/58241
- **V06:** https://github.com/vllm-project/vllm/issues/61927
- **V07:** https://github.com/vllm-project/vllm/issues/60174
- **V08:** https://github.com/vllm-project/vllm/pull/55601
- **V09:** https://github.com/vllm-project/vllm/issues/55775
- **V10:** https://github.com/vllm-project/vllm/issues/61738
- **V11:** https://github.com/vllm-project/vllm/issues/53504
- **V12:** https://github.com/vllm-project/vllm/pull/48375
- **V13:** https://docs.vllm.ai/en/latest/serving/data_parallel_deployment/
- **V14:** https://github.com/vllm-project/vllm/issues/50681
- **V15:** https://github.com/vllm-project/vllm/pull/56085
- **V16:** https://github.com/vllm-project/vllm/issues/56493
- **V17:** https://github.com/vllm-project/vllm/issues/60262
- **N01:** https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting/gpu_troubleshooting.html
- **N02:** https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md
- **N03:** https://developer.nvidia.com/blog/co-designing-ai-models-using-speculative-decoding-for-faster-llm-inference

- **N04:** https://www.nvidia.com/en-in/data-center/l4/
- **M01:** https://huggingface.co/incoai/Qwen3.8-27B-DFlash2
- **M02:** https://huggingface.co/z-lab/Qwen3.8-27B-DFlash2
- **M03:** https://inco.ai/blog/dflash2/
- **K01:** https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/overview/upgraded-accelerators

Transfer notes for Claude / another auditor

- Read this README **before** any GPU execution. Primary correctness links: S04, S05, S03, S01, V01, V02, V04, S07, V05.
- Do not infer merged status from PR title/issue `Closed`; inspect installed wheel/commit and release notes.
- For each claimed fix report source `URL`, `issue status`, `first broken release`, `fix commit`, `included in exact pinned version?`, exact reproduction and confidence. If no evidence, write **NOT FOUND**.
- Do not turn hypothetical acceptance/speed/memory figures into measured facts.
- For every blocker return **minimal patch + expected failure before + regression test after**.
- Never run Kaggle GPU or push a production notebook before user approval; this report makes no such claim.